Chapter 22: Stored Procedures

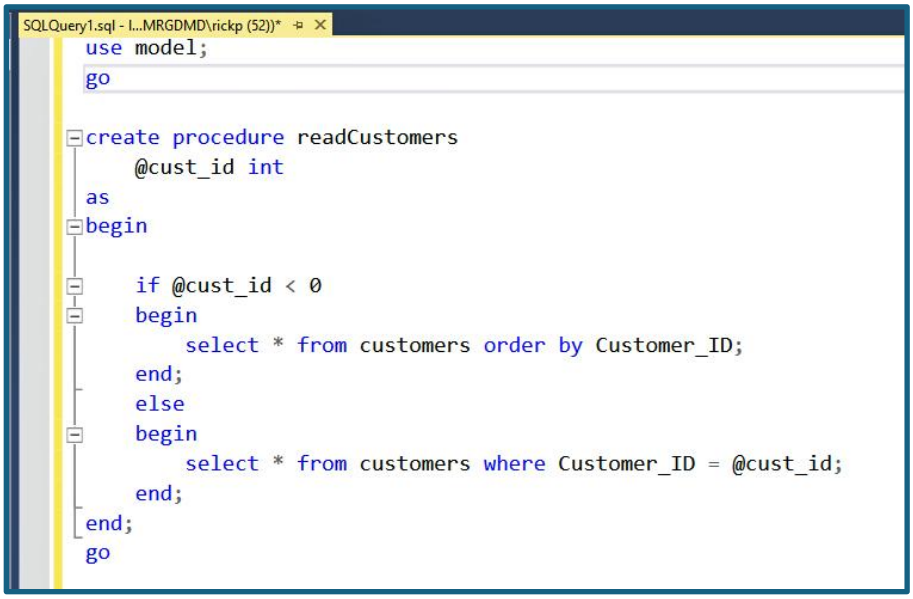
In the previous chapter we embedded our SQL within our C# code. Perfectly acceptable methodology if your application doesn't have to scale up for many users. If however you are going to have a large number of users, then you want to move that SQL inside the database in the form of a stored procedure.

SQL Server has an internal programming language called T-SQL for writing simple stored procedures. Oracle has a much more robust internal programming language called PL/SQL. Yes, for those of you who are interested, I have several books and courses on PL/SQL – unabashed advertisement there sorry.

In this lesson we are going to learn how to write stored procedures using Microsoft's T-SQL. Let's jump to the inside and take a look at some code.

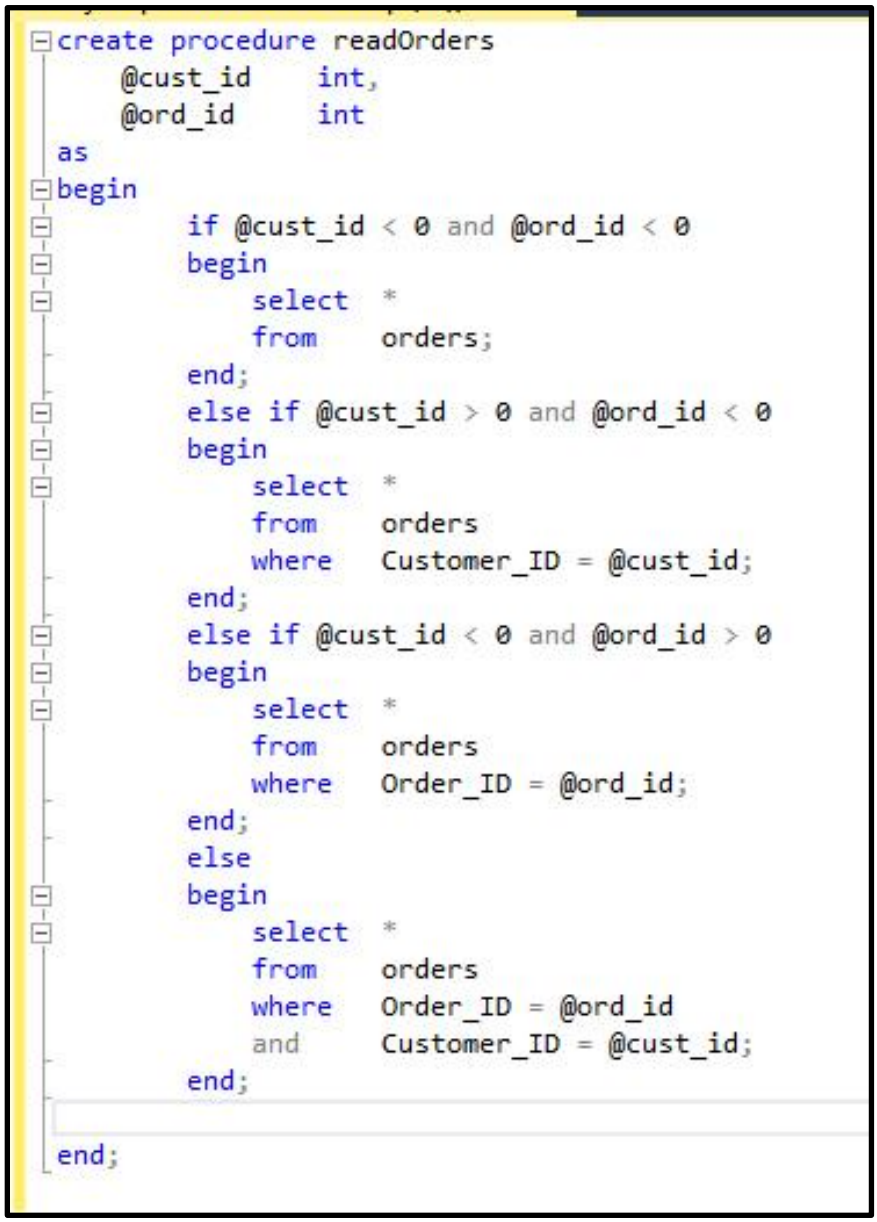
We begin with the code shown here which of course was written in SQL Server Management Studio, SSMS. (Figure 22.1) The keywords create procedure are self explanatory. ReadCustomers is the name of this procedure and is how we will reference it from within our C# code. The @cust_id int says this routine accepts one input parameter named cust_id of type integer. The rest is pretty self explanatory except maybe for the begin – end combinations which act to delineate each block of code.

Figure 21.1: readCustomers



The code for the readOrders (Figure 22.2) and readLineItems (Figure 22.3) looks similar so go ahead and knock those guys out too. The only difference is in the readLineItems routine we denormalize the query to include the product name field.

Figure 22.3: readOrders



```
create procedure readOrders
    @cust_id    int,
    @ord_id     int
as
begin
        if @cust_id < 0 and @ord_id < 0
        begin
            select  *
            from    orders;
        end;
        else if @cust_id > 0 and @ord_id < 0
        begin
            select  *
            from    orders
            where   Customer_ID = @cust_id;
        end;
        else if @cust_id < 0 and @ord_id > 0
        begin
            select  *
            from    orders
            where   Order_ID = @ord_id;
        end;
        else
        begin
            select  *
            from    orders
            where   Order_ID = @ord_id
            and     Customer_ID = @cust_id;
        end;

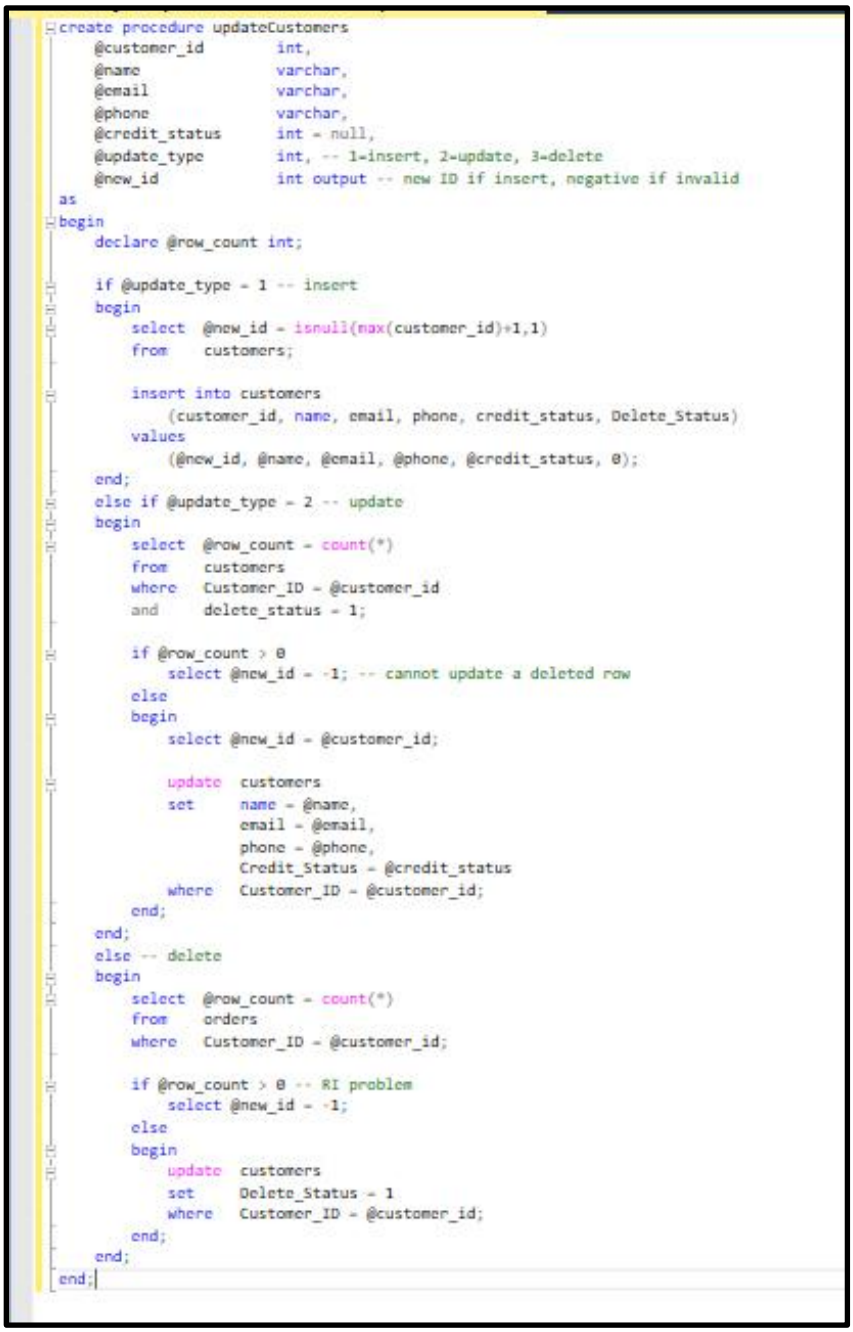
end;
```

Figure 22.3: readLineItems

```
create procedure readLineItems
    @ord_id     int
as
begin
    if @ord_id < 0
    begin
        select  order_id, line_item_number, a.product_id, quantity,
                extended_price, b.name
        from    Line_Items a, products b
        where   a.Product_ID = b.Product_ID;
    end;
    else
    begin
        select  order_id, line_item_number, a.product_id, quantity,
                extended_price, b.name
        from    Line_Items a, products b
        where   a.Product_ID = b.Product_ID
        and     a.Order_ID = @ord_id;
    end;
end;
```

Next comes our updateCustomers routine where the code gets considerably more interesting. First, note the @new_id parameter as type int output. This parameter will be returned to our C# code. If the routine does an insert then this parameter will contain the customer_id of the new row. If this parameter is negative then the update or delete was invalid. Next, please note how we declare a local variable named @row_count. That guy will be used in the select count syntax later in the routine. Finally, please note how we don't need to delineate single commands with the begin-end syntax. (Figure 22.4)

Figure 22.4: updateCustomers



```
create procedure updateCustomers
    @customer_id        int,
    @name               varchar,
    @email              varchar,
    @phone              varchar,
    @credit_status      int = null,
    @update_type        int, -- 1=insert, 2=update, 3=delete
    @new_id             int output -- new ID if insert, negative if invalid
as
begin
    declare @row_count int;

    if @update_type = 1 -- insert
    begin
        select  @new_id = isnull(max(customer_id)+1,1)
        from    customers;

        insert into customers
            (customer_id, name, email, phone, credit_status, Delete_Status)
        values
            (@new_id, @name, @email, @phone, @credit_status, 0);
    end;
    else if @update_type = 2 -- update
    begin
        select  @row_count = count(*)
        from    customers
        where   Customer_ID = @customer_id
        and     delete_status = 1;

        if @row_count > 0
            select @new_id = -1; -- cannot update a deleted row
        else
        begin
            select @new_id = @customer_id;

            update  customers
            set     name = @name,
                    email = @email,
                    phone = @phone,
                    Credit_Status = @credit_status
            where   Customer_ID = @customer_id;
        end;
    end;
    else -- delete
    begin
        select  @row_count = count(*)
        from    orders
        where   Customer_ID = @customer_id;

        if @row_count > 0 -- RI problem
            select @new_id = -1;
        else
        begin
            update  customers
            set     Delete_Status = 1
            where   Customer_ID = @customer_id;
        end;
    end;
end;
```

My updateOrders routine looks just the same except it has some date arithmetic. As I have mentioned previously, all dates are stored as the number of ticks plus or minus some reference point. Given that a date is really only an integer, date arithmetic is super easy behind the scenes. Here in this routine we use the dateadd function to add 30 days to the current date which is obtained via the getdate function. (Figure 22.5)

Figure 22.5: updateOrders

```
create procedure updateOrders
    @order_id       int,
    @order_status   varchar,
    @customer_id    int,
    @update_type    int, -- 1=insert, 2=update, 3=delete
    @new_id         int output -- new ID if insert, negative if invalid
as
begin
    declare @row_count int;
    declare @duedate datetime;

    if @update_type = 1 -- insert
    begin
        select @new_id = isnull(max(order_id)+1,1)
        from    Orders;

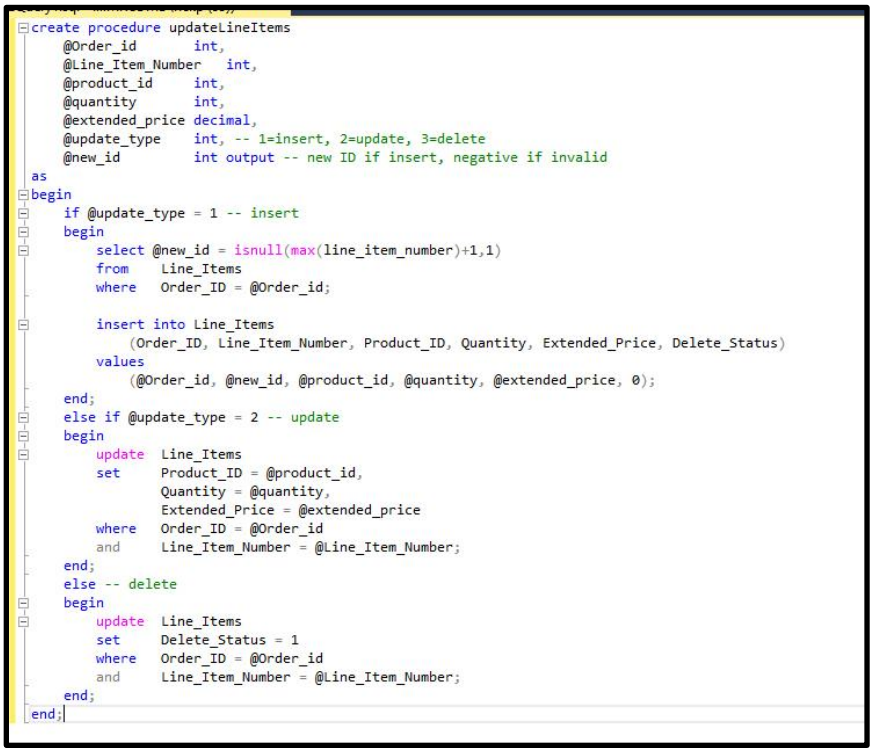
        select @duedate = DATEADD(dd, 30, getdate());
        insert into Orders
            (order_id, order_status, Order_Date, Due_Date, Customer_ID, Delete_Status)
        values
            (@new_id, @order_status, GETDATE(), @duedate, @customer_id, 0);
    end;
    else if @update_type = 2 -- update
    begin
        select  @row_count = count(*)
        from    Line_Items
        where   Order_ID = @order_id;

        if @row_count > 0
            select @new_id = -1; -- cannot do the update, brake RI
        else
        begin
            select  @new_id = @order_id;
            update  orders
            set     Order_Status = @order_status,
                    Customer_ID = @customer_id
            where   Order_ID = @order_id;
        end;
    end;
    else -- delete
    begin
        select  @row_count = count(*)
        from    Line_Items
        where   Order_ID = @order_id;

        if @row_count > 0
            select @new_id = -1; -- cannot do the update, brake RI
        else
        begin
            update  orders
            set     Order_Status = @order_status,
                    customer_id = @customer_id
            where   Order_ID = @order_id;
        end;
    end;
end;
```

The updateLineItems routine is again very similar, but we don't have to worry about breaking referential integrity constraints this time because we are at a leaf node in our E-R diagram. (Figure 22.6)

Figure 22.6: updateLineItems



```
create procedure updateLineItems
    @Order_id       int,
    @Line_Item_Number   int,
    @product_id     int,
    @quantity       int,
    @extended_price decimal,
    @update_type    int, -- 1=insert, 2=update, 3=delete
    @new_id         int output -- new ID if insert, negative if invalid
as
begin
    if @update_type = 1 -- insert
    begin
        select @new_id = isnull(max(line_item_number)+1,1)
        from    Line_Items
        where   Order_ID = @Order_id;

        insert into Line_Items
            (Order_ID, Line_Item_Number, Product_ID, Quantity, Extended_Price, Delete_Status)
        values
            (@Order_id, @new_id, @product_id, @quantity, @extended_price, 0);
    end;
    else if @update_type = 2 -- update
    begin
        update  Line_Items
        set     Product_ID = @product_id,
                Quantity = @quantity,
                Extended_Price = @extended_price
        where   Order_ID = @Order_id
        and     Line_Item_Number = @Line_Item_Number;
    end;
    else -- delete
    begin
        update  Line_Items
        set     Delete_Status = 1
        where   Order_ID = @Order_id
        and     Line_Item_Number = @Line_Item_Number;
    end;
end;
```

Okay, that concludes the creation of our stored procedures within SQL Server. Now it is time to open up Visual Studio and start to call some of those same procedures from within C#.

We begin by building the same GUI as we used in the last lesson wherein we have three related DataGridViews for our customers, orders and lineitems. (Figure 22.7)

Figure 22.7: GUI

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    this.Size = new System.Drawing.Size(1200, 800);
    this.Text = "Customer Service";

    DataGridView customersGridView = new DataGridView();
    customersGridView.Name = "customersGridView";
    customersGridView.Location = new System.Drawing.Point(25, 25);
    customersGridView.BorderStyle = BorderStyle.FixedSingle;
    customersGridView.Size = new System.Drawing.Size(550, 250);
    this.Controls.Add(customersGridView);

    DataGridView ordersGridView = new DataGridView();
    ordersGridView.Name = "ordersGridView";
    ordersGridView.Location = new System.Drawing.Point(650, 25);
    ordersGridView.BorderStyle = BorderStyle.FixedSingle;
    ordersGridView.Size = new System.Drawing.Size(500, 250);
    this.Controls.Add(ordersGridView);

    DataGridView lineitemsGridView = new DataGridView();
    lineitemsGridView.Name = "lineitemsGridView";
    lineitemsGridView.Location = new System.Drawing.Point(25, 300);
    lineitemsGridView.BorderStyle = BorderStyle.FixedSingle;
    lineitemsGridView.Size = new System.Drawing.Size(1125, 350);
    this.Controls.Add(lineitemsGridView);

    Button submitButton = new Button();
    submitButton.Name = "submitButton";
    submitButton.BackColor = System.Drawing.Color.Red;
    submitButton.Location = new System.Drawing.Point(450, 675);
    submitButton.Size = new System.Drawing.Size(250, 50);
    submitButton.Text = "Submit";
    this.Controls.Add(submitButton);
}
```

Next we populate our DataTables through our read stored procedures. Please note how we had to manually open our connection this time in order to utilize the stored procedure SQLCommand CommandType method. This syntax is a little slippery but once you get the first one working it all comes together pretty quickly. (Figure 22.8)

Figure 22.8: Populate DataTables Via Read Stored Procedures

```
submitButton.Location = new System.Drawing.Point(450, 675);
submitButton.Size = new System.Drawing.Size(250, 50);
submitButton.Text = "Submit";
this.Controls.Add(submitButton);

SqlConnection conn = new SqlConnection(@"Data Source=localhost\SQLEXPRESS; Initial Catalog = model;
                                         Integrated Security = True; TrustServerCertificate = True;");
conn.Open();

myCommand = new SqlCommand("readCustomers", conn);
myCommand.CommandType = CommandType.StoredProcedure;
myCommand.Parameters.AddWithValue("cust_id", -1);  // read all the customer rows
myDA = new SqlDataAdapter(myCommand);
myDA.Fill(tblCustomers);

myCommand = new SqlCommand("readOrders", conn);
myCommand.CommandType = CommandType.StoredProcedure;
myCommand.Parameters.AddWithValue("cust_id", -1);
myCommand.Parameters.AddWithValue("ord_id", -1);
myDA = new SqlDataAdapter(myCommand);
myDA.Fill(tblOrders);

myCommand = new SqlCommand("readLineItems", conn);
myCommand.CommandType = CommandType.StoredProcedure;
myCommand.Parameters.AddWithValue("ord_id", -1);
myDA = new SqlDataAdapter(myCommand);
myDA.Fill(tblLineItems);
```

It was of course time then to set up our relationships between those grids. Same methods as we used in the last chapter it is only the way we got the data out of the database that changed. (Figure 22.9)

Figure 22.9: Setup Grid Relationships

```
myCommand = new SqlCommand("readLineItems", conn);
myCommand.CommandType = CommandType.StoredProcedure;
myCommand.Parameters.AddWithValue("ord_id", -1);
myDA = new SqlDataAdapter(myCommand);
myDA.Fill(tblLineItems);

tblDataSet = new DataSet();
tblDataSet.Tables.Add(tblCustomers);
tblDataSet.Tables.Add(tblOrders);
tblDataSet.Tables.Add(tblLineItems);

tblDataSet.Relations.Add("Cust_Ord_Relation", tblCustomers.Columns["customer_id"], tblOrders.Columns["customer_id"]);
tblDataSet.Relations.Add("Ord_Line_Relation", tblOrders.Columns["order_id"], tblLineItems.Columns["order_id"]);

BindingSource bsCust = new BindingSource();
bsCust.DataSource = tblDataSet;
bsCust.DataMember = "tblCustomers";

BindingSource bsOrd = new BindingSource();
bsOrd.DataSource = bsCust;
bsOrd.DataMember = "Cust_Ord_Relation";

BindingSource bsLine = new BindingSource();
bsLine.DataSource = bsOrd;
bsLine.DataMember = "Ord_Line_Relation";

customersGridView.DataSource = bsCust;
ordersGridView.DataSource = bsOrd;
lineitemsGridView.DataSource = bsLine;
```

Time to work on that submit button Click event handler. In an effort to keep this lesson at a reasonable length, I will only show you the updateCustomers Add routine here but I am sure you can write the rest given this example code. (Figure 22.10) Please note how stored procedure output parameters are handled a little bit differently from input parms.

Figure 22.10: Submit Click



```
0 references
private void SubmitButton_Click(object sender, EventArgs e)
{

    SqlConnection conn = new SqlConnection(@"Data Source=localhost\SQLEXPRESS; Initial Catalog = model;
                                             Integrated Security = True; TrustServerCertificate = True;");
    conn.Open();

    myCommand = new SqlCommand("updateCustomers", conn);
    myCommand.CommandType = CommandType.StoredProcedure;

    foreach (DataRow myRow in tblCustomers.Rows)
    {
        // we will do inserts first
        if (myRow.RowState == DataRowState.Added)
        {
            myCommand.Parameters.AddWithValue("@customer_id", 0); // new customer_id will come back in @new_id
            myCommand.Parameters.AddWithValue("@name", myRow["name"]);
            myCommand.Parameters.AddWithValue("@email", myRow["email"]);
            myCommand.Parameters.AddWithValue("@phone", myRow["phone"]);
            myCommand.Parameters.AddWithValue("@credit_status", myRow["credit_status"]);
            myCommand.Parameters.AddWithValue("@update_type", 1);
            myCommand.Parameters.Add("@new_id", SqlDbType.Int).Direction = ParameterDirection.Output;

            myCommand.ExecuteNonQuery();

            if (myCommand.Parameters["@new_id"].Value.ToString() == "-1") // something went wrong
                MessageBox.Show("Customer " + myRow["name"] + " failed to insert, please check the data.");
            else
                myRow["customer_id"] = (int)myCommand.Parameters["@new_id"].Value;

            myCommand.Parameters.Clear();
        }
    }
```

Okay, that puts a wrap on our stored procedure discussion. In our next lesson we are going to take a look at XML, the Extensible Markup Language. Whereas a relational database is the right place to permanently store your data, XML files make a great place for temporary data storage.